LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 2

Stack dan Queue

Oleh:

Ahmad Riyadhi **NIM. 2510817110004**

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

MARET 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 1

Laporan Praktikum Algoritma & Struktur Data Modul 1: Struct & Pointer ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Prakitkum ini dikerjakan oleh:

Nama Praktikan : Ahmad Riyadhi
NIM : 2510817110004

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Fadhil Syahdama Mahatma Putra NIM. 2410817210026	Andreyan Rizky Baskara, S.Kom., M.Kom NIP. 199307032019031011

DAFTAR ISI

Contents

LEMBAR PENGESAHAN 1
DAFTAR ISI 2
DAFTAR GAMBAR 3
DAFTAR TABEL 4
SOAL 1 1
A. Pembahasan 1
SOAL 2 3
A. Source Code 4
B. Output Program 6
C. Pembahasan 8
SOAL 3 12
A. Source Code 13
B. Output Program 15
C. Pembahasan 16
LINK GITHUB 20

DAFTAR GAMBAR

Gambar 1 Tampilan Source Code Soal 2 3
gambar 2 Screenshot tampilan Awal Program Saat Dijalankan 6
gambar 3 Screenshot Output Program Soal 2 "Proses input data ke dalam stack" 6
gambar 4 Screenshot Output Program Soal 2 "Tampilan isi stack 6
gambar 5 Screenshot Output Program Soal 2 "Proses penghapusan data" 7
gambar 6 Screenshot Output Program Soal 2 "Proses penghapusan data" 7
gambar 7 Tampilan Source Code Soal 3 12
gambar 8 Screenshot Output Program Soal 3 "Tampilan Proses Input data" 15
gambar 9 Screenshot Output Program Soal 3 "Tampilan Cetak Queue" 15
gambar 10 Screenshot Output Program Soal 3 Tampilan Proses Delete & Cetak Queue kembali" ... 15
gambar 11 Screenshot Output Program Soal 3 "Tampilan Proses Delete Kembali & Kondisi Queue Kosong" 16

DAFTAR TABEL

Table 1 Source Code Soal Nomor 2 4
Table 2 Source Code Soal Nomor 3 13

SOAL 1

Apa perbedaan Stack dengan Queue?.

A. Pembahasan

Stack dan Queue pada dasarnya adalah dua jenis struktur data linier yang difungsikan untuk menyimpan dan mengelola koleksi data. Perbedaan paling mendasar di antara keduanya terletak pada mekanisme atau aturan akses elemennya (bagaimana alur data tersebut dimasukkan dan dikeluarkan dari memori).

Stack (Tumpukan)

- **Prinsip Kerja:** Beroperasi menggunakan logika **LIFO (Last In, First Out)**. Artinya, elemen data yang paling terakhir disisipkan ke dalam memori akan menjadi elemen yang pertama kali dieksekusi atau dikeluarkan.
- **Operasi Utama:** Penambahan elemen baru disebut dengan operasi **Push**, sedangkan penghapusan elemen disebut **Pop**. Kedua operasi ini dilakukan hanya melalui satu pintu yang sama, yaitu ujung atas tumpukan (Top).
- **Analogi Logis:** Konsep ini identik dengan tumpukan piring kotor di wastafel. Piring yang terakhir kali ditumpuk di posisi teratas pasti akan menjadi piring pertama yang diambil untuk dicuci.
- **Contoh Penggunaan:** Sering diimplementasikan pada sistem Call Stack pemanggilan fungsi di memori komputer, mekanisme Undo pada aplikasi, atau riwayat mundur (Back) pada browser.

Queue (Antrean)

- **Prinsip Kerja:** Mengadaptasi logika **FIFO (First In, First Out)**. Hal ini berarti elemen data yang pertama kali masuk ke dalam sistem akan mendapatkan prioritas utama untuk diproses dan dikeluarkan lebih awal.

- **Operasi Utama:** Memasukkan data ke dalam sistem dinamakan **Enqueue** (di ujung belakang/Rear), sedangkan proses mengambil data disebut **Dequeue** (di ujung depan/Front). Berbeda dengan Stack, antrean ini terjadi pada dua ujung yang berlawanan.
- **Analogi Logis:** Konsep ini secara nyata persis seperti antrean pembeli di kasir restoran. Pelanggan yang datang lebih awal untuk mengantre di depan tentu akan dilayani dan selesai terlebih dahulu.
- **Contoh Penggunaan:** Relevan digunakan untuk antrean pencetakan dokumen pada printer (Print Spooler), penjadwalan antrean proses di CPU, atau pengelolaan bandwidth jaringan.

SOAL 2

Cobalah contoh program berikut, running dan analisis hasilnya!

```
int penuh()
{
    if(Tumpuk.atas == max-1)
        return 1;
    else
        return 0;
}

void input (int data)
{
    if (kosong()==1)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else if(penuh()==0)
    {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data "<< Tumpuk.data[Tumpuk.atas]
             << " Masuk Ke Stack ";
    }
    else
        cout << "Tumpukan Penuh";
}
void hapus()
{
    if(kosong()== 0)
    {
        cout << "Data Teratas Sudah Terambil";
        Tumpuk.atas--;
    }
    else
    cout <<" Data Kosong";
}
void tampil()
{
    if (kosong()== 0)
    {
        for(int i = Tumpuk.atas; i>=0; i--)
        {
            cout <<"\nTumpukan Ke " << i << " = "
            <<Tumpuk.data[i];
        }
    }
    else
    cout << "Tumpukan Kosong";
}
void bersih ()
{
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong !";
}
```

Gambar 1 Tampilan Source Code Soal 2

A. Source Code

Table 1 Source Code Soal Nomor 2

```
#include <iostream>
using namespace std;
#define max 5

// Struktur Stack
struct Stack {
    int data[max];
    int atas;
};

Stack Tumpuk;

// Fungsi cek kosong
int kosong() {
    if (Tumpuk.atas == -1)
        return 1;
    else
        return 0;
}

// Fungsi cek penuh
int penuh() {
    if (Tumpuk.atas == max - 1)
        return 1;
    else
        return 0;
}

// Fungsi input (push)
void input(int data) {
    if (kosong() == 1) {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas] << "
Masuk Ke Stack\n";
    } else if (penuh() == 0) {
        Tumpuk.atas++;
        Tumpuk.data[Tumpuk.atas] = data;
        cout << "Data " << Tumpuk.data[Tumpuk.atas] << "
Masuk Ke Stack\n";
    } else {
        cout << "Tumpukan Penuh\n";
    }
}

// Fungsi hapus (pop)
void hapus() {
    if (kosong() == 0) {
        cout << "Data Teratas Sudah Terambil\n";
```

```
        Tumpuk.atas--;
    } else {
        cout << "Data Kosong\n";
    }
}

// Fungsi tampil
void tampil() {
    if (kosong() == 0) {
        for (int i = Tumpuk.atas; i >= 0; i--) {
            cout << "Tumpukan Ke " << i << " = " <<
Tumpuk.data[i] << endl;
        }
    } else {
        cout << "Tumpukan Kosong\n";
    }
}

// Fungsi bersih
void bersih() {
    Tumpuk.atas = -1;
    cout << "Tumpukan Kosong!\n";
}

// Program utama
int main() {
    Tumpuk.atas = -1; // inisialisasi stack
    int pilihan, data;
    do {
        cout << "\n=== MENU STACK ===\n";
        cout << "1. Input (Push)\n";
        cout << "2. Hapus (Pop)\n";
        cout << "3. Tampil\n";
        cout << "4. Bersih\n";
        cout << "5. Keluar\n";
        cout << "Pilih: ";
        cin >> pilihan;

        switch (pilihan) {
            case 1:
                cout << "Masukkan data: ";
                cin >> data;
                input(data);
                break;
            case 2:
                hapus();
                break;
            case 3:
                tampil();
                break;
            case 4:
```

```
                    bersih();
                    break;
            }
        } while (pilihan != 5);

        return 0;
}
```

B. Output Program

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Windows 11> cd 'c:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output'
PS C:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output> & .\'PRAK201-2510817110004-AhmadRiyadhi.exe'

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih:
```

gambar 2 Screenshot tampilan Awal Program Saat Dijalankan

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Windows 11> cd 'c:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output'
PS C:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output> & .\'PRAK201-2510817110004-AhmadRiyadhi.exe'

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 1
Masukkan data: 10
Data 10 Masuk Ke Stack

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 20

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 30
```

gambar 3 Screenshot Output Program Soal 2 "Proses input data ke dalam stack"

```
=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 3
Tumpukan Ke 2 = 30
Tumpukan Ke 1 = 20
Tumpukan Ke 0 = 10
```

gambar 4 Screenshot Output Program Soal 2 "Tampilan isi stack

```
=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 2
Data Teratas Sudah Terambil
```

gambar 5 Screenshot Output Program Soal 2 "Proses penghapusan data"

```
=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 4
Tumpukan Kosong!

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 3
Tumpukan Kosong

=== MENU STACK ===
1. Input (Push)
2. Hapus (Pop)
3. Tampil
4. Bersih
5. Keluar
Pilih: 5
PS C:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output>
```

gambar 6 Screenshot Output Program Soal 2 "Proses penghapusan data"

.

C. Pembahasan

Kode di atas merupakan realisasi dari struktur data Stack (Tumpukan) yang dibangun di atas fondasi *Array*. Seluruh siklus data pada program ini dikontrol secara ketat oleh hukum LIFO (*Last In, First Out*). Melalui sebuah menu *Command Line Interface* (CLI), program ini menuntun pengguna untuk bisa mengeksekusi empat perintah mendasar: menyisipkan elemen (push), membuang elemen teratas (pop), merender urutan elemen (tampil), dan mereset total *array* (bersih).

Mekanisme penunjuk (*pointer*) diatur oleh variabel Tumpuk.atas. Saat fungsi input() dijalankan, nilai indeks penunjuk akan naik, lalu elemen data akan disematkan di sana, selama memori belum mencapai batasnya. Jika hapus() ditekan, program membiarkan fisik data berada di *array* namun "membutakan" pembacaan sistem dengan cara mengurangi angka Tumpuk.atas.

Saat dieksekusi, menu tampil() akan mengurai elemen dari atas ke bawah (dari indeks terbesar menuju 0), sehingga bukti validasi prinsip LIFO terlihat jelas. Sebaliknya, fitur bersih() menawarkan jalan instan untuk memformat tumpukan dengan mengunci mati indikator Tumpuk.atas kembali ke angka -1.

Analisis Baris Kode (Syntax Code):

1. #include <iostream>: Mengaktifkan fungsionalitas pustaka standar *input/output* agar program mengenali operasi cin dan cout.
2. using namespace std;: Memperpendek penulisan perintah standar sehingga kita dibebaskan dari kewajiban mengetik awalan std:: pada fungsi bawaan C++.
3. #define max 5: Menciptakan sebuah konstanta paten pembatas kapasitas. Stack ini secara mutlak hanya mampu memuat paling banyak 5 elemen data.

4. struct Stack { ... }: Membuat kerangka data kustom bernama Stack untuk menyatukan dua variabel: tempat menampung *array* dan indikator posisi elemen tertinggi.
5. int data[max];: Alokasi memori fisik berupa *array* tipe *integer* yang sanggup diisi data sebanyak limit max.
6. int atas;: Deklarasi variabel pembidik indeks paling atas (*top*). Variabel ini sangat dinamis mengikuti jumlah elemen.
7. Stack Tumpuk;: Inisiasi pembentukan objek bernama Tumpuk yang mengadopsi format struct Stack.
8. int kosong() { ... }: Fungsi spesifik yang memeriksa ada tidaknya muatan di dalam *stack*.
9. if(Tumpuk.atas == -1): Jika posisi indikator adalah -1, ini menjadi sinyal mutlak bahwa wadah penyimpanan sama sekali belum tersentuh.
10. return 1;: Mengirimkan status kebenaran bahwa *stack* memang kosong.
11. else return 0;: Merupakan nilai balasan (*return*) ketika memori dipastikan memiliki minimal 1 buah elemen di dalamnya.
12. int penuh() { ... }: Fungsi verifikator untuk mendeteksi apakah wadah memori telah sesak dan kehabisan ruang.
13. if(Tumpuk.atas == max-1): Mengecek apabila indikator atas telah menyamai batas maksimal indeks array (angka 4).
14. return 1;: Mengirim balik sinyal *true* bahwa kapasitas penuh.
15. void input (int data): Prosedur eksekusi *Push* untuk menginjeksi sebuah elemen baru.
16. if (kosong() == 1): Kondisi awal ketika hendak memasukkan elemen perdana.
17. Tumpuk.atas++;: Posisi indeks yang sebelumnya -1 akan dimajukan menjadi 0.
18. Tumpuk.data[Tumpuk.atas] = data;: Menyalin angka yang diketik pengguna ke dalam *array* persis di indeks tersebut.
19. else if(penuh()==0): Kondisi opsional ketika wadah sudah ada isinya namun belum mencapai titik kapasitas.

20. Tumpuk.atas++; Tumpuk.data[...] = data;: Menaikkan lagi penunjuk *array*, lalu menjejalkan data ke dalamnya.
21. else cout << "Tumpukan Penuh\n";: Proteksi agar program tidak rusak (*overflow*). Jika sudah sesak, pengguna ditolak saat melakukan *Push*.
22. void hapus(): Prosedur eksekusi *Pop* untuk memotong elemen di posisi puncak.
23. if(kosong() == 0): Operasi pencabutan hanya diizinkan jika secara riil ada data di memori.
24. Tumpuk.atas--;: Logika pembersihan; kita hanya menurunkan posisi indikator sehingga data di atasnya sudah tidak diakui lagi.
25. else cout <<"Data Kosong\n";: Pencegahan error agar program tidak memaksa turun dari titik -1 saat memori masih kosong.
26. void tampil(): Sub-program untuk mencetak visualisasi data.
27. if(kosong() == 0): Mengecek dulu sebelum mencetak. Jika ada data, baru iterasi berjalan.
28. for(int i = Tumpuk.atas; i>=0; i--): Perulangan *for* yang dipaksa berjalan mundur dari posisi tinggi (*Top*) menjalar pelan-pelan ke dasar *array* (0).
29. cout << ...: Menampilkan indeks sekaligus angka di layar terminal.
30. else cout << "Tumpukan Kosong\n";: Konfirmasi *output* bila memang tak ada elemen yang pantas dicetak.
31. void bersih (): Fitur jalan pintas untuk membasmi seluruh jejak elemen.
32. Tumpuk.atas = -1;: Indikator ditarik paksa ke -1, sebuah manipulasi yang seketika menjadikan tumpukan kembali suci dan kosong.
33. cout << "Tumpukan Kosong!\n";: Pemberitahuan di layar bahwa perintah *reset* sukses.
34. int main(): Jantung dari eksekusi program. Kompiler C++ selalu memulai proses dari blok ini.
35. Tumpuk.atas = -1;: Penetapan konfigurasi *default* sebelum menu berjalan.
36. int pilihan, data;: Menyiapkan variabel lokal untuk menampung ketikan respons pengguna dan memori sementara angkanya.

37. do { ...: Membuka siklus perulangan do-while agar aplikasi CLI tidak serta merta tertutup seusai satu operasi saja.
38. cin >> pilihan;: Program berhenti sejenak, mengamati angka berapakah yang ditekan pengguna di *keyboard*.
39. switch(pilihan) { ... }: Pintu pengatur lalu lintas komputasi. Jika input 1, arahkan ke fungsi penambah data, jika 2 arahkan ke fungsi pembuang data, dan seterusnya.
40. } while(pilihan != 5);: Selama variabel pilihan bukan angka 5, blok perintah do akan senantiasa diputar ulang tiada henti.
41. return 0;: Garis akhir berjalannya fungsi utama. Menyatakan bahwa program telah tuntas tanpa kegagalan.

SOAL 3

Cobalah contoh program berikut, running dan analisis hasilnya!

```
#include<iostream>
#include<conio.h>
#include<stdlib.h>
#define n 10
using namespace std;

void INSERT();
void DELETE();
void CETAKLAYAR();
void Inisialisasi();
void RESET();
int PIL,F,R;
char PILIHAN [1],HURUF;
char Q[n];
int main ( )
{
     Inisialisasi();
     do
     {
          cout<<"QUEUE"<<endl;
          cout<<"==============="<<endl;
          cout<<"1. INSERT"<<endl;
          cout<<"2. DELETE"<<endl;
          cout<<"3. CETAK QUEUE"<<endl;
          cout<<"4. QUIT"<<endl;
          cout<<"PILIHAN : "; cin>>PILIHAN;
          PIL=atoi(PILIHAN);
      switch (PIL)
      {
      case 1:
           INSERT ();
           break;
      case 2:
           DELETE();
           break;
      case 3:
           CETAKLAYAR ();
           break;
      default:
           cout<<"TERIMA KASIH"<<endl;
           break;
      }
      cout<<"press any key to continue"<<endl;
      getch();
      system("cls");
     }
     while (PIL<4);
}
```

gambar 7 Tampilan Source Code Soal 3

A. Source Code

Table 2 Source Code Soal Nomor 3

```
#include<iostream>
#include<stdlib.h>
using namespace std;
#define n 10

int F = -1, R = -1;
char Q[n];
int PIL;

bool kosong() {
    return (F == -1);
}

bool penuh() {
    return (R == n - 1);
}

void Inisialisasi() {
    F = R = -1;
}

void INSERT() {
    if (penuh()) {
        cout << "Queue Penuh!\n";
    } else {
        char data;
        cout << "Masukkan data: ";
        cin >> data;
        if (kosong()) {
            F = R = 0;
        } else {
            R++;
        }
        Q[R] = data;
        cout << "Data masuk ke queue\n";
    }
}

void DELETE() {
    if (kosong()) {
        cout << "Queue Kosong!\n";
    } else {
        cout << "Data keluar: " << Q[F] << endl;
        if (F == R) {
            F = R = -1;
        } else {
            F++;
        }
```

```
    }
}

void CETAKLAYAR() {
    if (kosong()) {
        cout << "Queue kosong\n";
    } else {
        cout << "Isi Queue: ";
        for (int i = F; i <= R; i++) {
            cout << Q[i] << " ";
        }
        cout << endl;
    }
}

void RESET() {
    F = R = -1;
    cout << "Queue dikosongkan\n";
}

int main() {
    Inisialisasi();
    do {
        cout << "\nQUEUE\n";
        cout << "==============\n";
        cout << "1. INSERT\n";
        cout << "2. DELETE\n";
        cout << "3. CETAK QUEUE\n";
        cout << "4. QUIT\n";
        cout << "PILIHAN : ";
        cin >> PIL;

        switch (PIL) {
            case 1:
                INSERT();
                break;
            case 2:
                DELETE();
                break;
            case 3:
                CETAKLAYAR();
                break;
            case 4:
                cout << "TERIMA KASIH\n";
                break;
            default:
                cout << "Pilihan tidak valid\n";
        }
    } while (PIL != 4);

    return 0;
}
```

B. Output Program

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Windows 11> cd 'c:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output'
PS C:\Users\Windows 11\OneDrive\Documents\kuliah\prak algo\modul 2\output> & .\'PRAK202-2510817110004-AhmadRiyadhi.exe'

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 1
Masukkan data: A
Data masuk ke queue

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 1
Masukkan data: B
Data masuk ke queue

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 1
Masukkan data: C
Data masuk ke queue
```

gambar 8 Screenshot Output Program Soal 3 "Tampilan Proses Input data"

```
QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 3
Isi Queue: A B C
```

gambar 9 Screenshot Output Program Soal 3 "Tampilan Cetak Queue"

```
QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Data keluar: A

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 3
Isi Queue: B C
```

gambar 10 Screenshot Output Program Soal 3 Tampilan Proses Delete & Cetak Queue kembali"

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Data keluar: B

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Data keluar: C

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Queue Kosong!

QUEUE
==============
1. INSERT
2. DELETE
3. CETAK QUEUE
4. QUIT
PILIHAN : 2
Queue Kosong!
```

gambar 11 Screenshot Output Program Soal 3 "Tampilan Proses Delete Kembali & Kondisi Queue Kosong"

C. Pembahasan

Berdasarkan hasil uji coba, kode program tersebut sanggup menyimulasikan sistem antrean (Queue) dengan sempurna menggunakan *array* berkapasitas sepuluh elemen. Arsitektur logika program ini tunduk sepenuhnya pada asas operasional FIFO (*First In First Out*).

Bisa kita lihat saat fungsi menu INSERT dipilih, data-data tambahan selalu menduduki posisi kosong yang berada paling buntut (*Rear*). Demikian juga saat fitur DELETE diuji, secara otomatis program akan mengeliminasi memori antrean dari posisi terdepan (*Front*).

Pengamanan program (*exception handling*) juga diolah secara rapi. Jika *user* gegabah menghapus antrean yang sejatinya sudah tak berisi, pesan "Queue Kosong" akan memblokir *error*. Pun jika ukuran *array* dijejali hingga indeks ke-9, fungsi pemasukan akan lumpuh

sementara berbarengan dengan munculnya teguran "Queue Penuh". Fungsi CETAKLAYAR juga dengan akurat merepresentasikan fisik memori dari titik *Front* ke *Rear*, menandakan sistem ini handal dalam mempertahankan keutuhan hierarki antrean.

Analisis Baris Kode (Syntax Code):

1. #include <iostream>: Mengakses berkas *header* standar untuk menyokong proses baca tulis data antarmuka, khususnya memanggil instruksi cout dan cin.
2. #include<stdlib.h>: *Header* bantuan C++ yang menyediakan berbagai *function* standar pengatur sistem, meski perannya pada kode ini difokuskan sebagai peredam intervensi kompilator tingkat rendah.
3. using namespace std;: Memungkinkan kita menggunakan perbendaharaan standar nama variabel dari *library* C++ secara spontan dan lebih ringkas.
4. #define n 10: Menjadikan huruf n sebagai angka mati 10. Menjamin batasan volume elemen antrean selalu seragam di seluruh cakupan program.
5. int F=-1, R=-1; char Q[n]; int PIL;:
    - F (Inisial dari *Front*): Pemandu titik koordinat elemen pertama.
    - R (Inisial dari *Rear*): Pemandu titik koordinat antrean paling buncit. Keduanya bertengger di -1 sebagai konfirmasi awal ketiadaan antrean.
    - Q[n]: Merupakan keranjang penyimpanan aktual berbentuk kumpulan memori karakter (*char* array).
    - PIL: Kotak wadah untuk menyimpan angka sakelar menu dari *user*.
6. bool kosong() { return (F==-1); }: Fitur utilitas yang melakukan inspeksi. Jika F menempel di angka -1, ia memuntahkan nilai benar (*true*). Bila tidak, maka dinyatakan salah (*false*).
7. bool penuh() { return (R==n-1); }: Utilitas inspektorat batas antrean. Akan memberi sinyal bahaya (*true*) hanya apabila posisi ujung buritan (R) sudah mentok dengan batas *array* tertinggi (n-1).
8. void Inisialisasi() { F=R=-1; }: Algoritma fundamental untuk menstandarkan parameter posisi F maupun R. Memastikan tak ada sisa jejak memori asing yang mengganggu awal jalannya program.
9. void INSERT() { if (penuh()) { ... } else { ...: Pintu masuk data ke antrean. Sistem secara preventif melihat rambu-rambu dari *return* fungsi penuh() agar kapasitas memori tidak meluber melebihi daya tampung 10 data.

10. char data; cout << "Masukkan data: "; cin >> data;: Memesan wadah sementara bertipe karakter, lalu menantikan dan menerima huruf atau simbol dari input papan ketik.
11. if (kosong()) { F=R=0; } else { R++; }: Cabang logika cerdas. Jika ia adalah pendaftar pertama di antrean yang kosong, langsung dititihkan di kursi terdepan (F & R bernilai 0). Tapi jika antrean sudah jalan, cukup perpanjang buntutnya selangkah ke belakang (R++).
12. Q[R] = data;: Momentum fiksasi, data karakter resmi mengendap secara berurutan di relung *array* posisi R.
13. cout << "Data masuk ke queue\n";: Konfirmasi verbal bagi pengguna bahwa sirkulasi insersi sukses dijalankan.
14. void DELETE() { if (kosong()) { ... }: Mengeksekusi penarikan data dari gerbang utama. Tentunya dengan memvalidasi dulu apakah memang ada data yang bisa diambil.
15. else { cout << "Data keluar: " << Q[F] << endl;: Mencetak isi *array* berindeks F sebagai pembuktian visual bahwa data terdepanlah yang terbuang.
16. if (F==R) { F=R=-1; } else { F++; }: Tata cara pembongkaran memori. Kalau elemen yang barusan dihapus adalah sisa satu-satunya elemen, seluruh antrean dinetralkan ke -1. Apabila deretan antrean masih padat, cukup geser titik batas depan maju ke antrean kedua (F++).
17. void CETAKLAYAR() { if (kosong()) { ... }: Algoritma inspektorat visual. Sistem enggan mencetak urutan layar manakala deteksi menunjukkan muatan kosong.
18. else { cout << "Isi Queue: "; for (int i=F; i<=R; i++) { ... }: Menggambar struktur murni antrean memori dengan merangkai data melalui iterasi for. Ia disiplin menapaki dari elemen berstatus *Front* murni sampai tuntas ke *Rear*.
19. cout << endl; }: Memberi ketukan '*Enter*' transisi (*new line*) pasca cetakan selesai dirender.
20. void RESET() { F=R=-1; cout << ... }: Modul pamungkas pembunuh antrean. Tanpa ba-bi-bu, F dan R dibinasakan ke -1. Tidak peduli seberapa panjangnya data, semua dianggap menguap seketika.
21. int main() { Inisialisasi();: Rongga inti dari aplikasi. Menjadi rumah beroperasinya Inisialisasi() agar tatanan *array* dipoles bersih sebelum dibiarkan berinteraksi dengan pengguna.

22. do { cout << "\nQUEUE\n";: Merancang *template* perulangan tanpa henti yang mana bingkai layar menu operasional terus dipertahankan ketersediaannya usai *user* rampung mengeksekusi layanan.
23. cout << "==========\n";: Elemen dekorasi UI (*User Interface*) untuk mengarsir batasan antarmuka layar teks.
24. cout << "1. INSERT\n";: Etalase opsi pendaftaran entitas anyar untuk dirangkai memanjang ke dalam wadah Queue.
25. cout << "2. DELETE\n";: Etalase opsi pembatalan atau pelayanan, fungsinya untuk membedol data nomor satu di depan.
26. cout << "3. CETAK QUEUE\n";: Fitur pantau langsung agar pengguna tidak menerka-nerka susunan apa yang bersemayam di perut antrean *array*.
27. cout << "4. QUIT\n";: Akses tombol darurat pelarian jika rutinitas program sudah dianggap rampung oleh *user*.
28. cout << "PILIHAN : "; cin >> PIL;: Merekam hasil pijatan *keyboard* yang diketik dan menyimpan respons desimalnya memori sakelar PIL.
29. switch (PIL) {: Menyebarkan percabangan nasib rute kontrol. Layaknya jalan tol, nilai PIL akan diarahkan meluncur ke bilik case sasarannya secara presisi.
30. case 1: INSERT(); break;: Saluran menuju sub-program fungsi *Enqueue*. Perintah break meyakinkan agar tidak tumpah tertabrak ke perintah case 2.
31. case 2: DELETE(); break;: Saluran menuju sub-program fungsi *Dequeue*. Membunuh secara rasional indeks ke-0 yang bertransformasi.
32. case 3: CETAKLAYAR(); break;: Jendela penyiaran log antrean. Membongkar hasil *output* ke muka layar CLI.
33. case 4: cout << "TERIMA KASIH\n"; break;: Teks sambutan penutupan, sebatas kata-kata perpisahan dari aplikasi ke tuannya.
34. default: cout << "Pilihan tidak valid\n";: Filter pencegat bila ternyata pengguna usil memasukkan karakter yang tidak difasilitasi di daftar menu resmi.
35. } while (PIL != 4);: Syarat absolut mesin terus berdetak. Program sudi untuk mengulangi do dari awal hanya apabila PIL tidak menunjukkan niat bernilai empat.
36. return 0; }: Kodifikasi sukses bagi *Operating System*. Artinya *compiled code* sudah berhenti dengan paripurna tak bersisa.

LINK GITHUB

https://github.com/TebangLiar/Praktikum-Algoritma-P1-Modul1/tree/main